

Data Frames

Mario Morales

Contenido I

1 DataFrames (marcos de datos)

Section 1

DataFrames (marcos de datos)

Creación de DataFrames

Una forma de crear DataFrames en R es mediante la función `data.frame`, como se muestra a continuación

```
set.seed(1234)
# 10 datos simulados de una variable aleatoria binomial con n=20
x<-rbinom(10,20,0.4)
# vector de caracteres aleatorios
y<-sample(c("F","M"),10,replace=TRUE)
# vector logico
z<-runif(10)>0.5
```

A continuación creamos un DataFrame con estos tres vectores

```
datos=data.frame(x,y,z)
datos
```

	x	y	z
1	5	M	FALSE
2	9	M	FALSE
3	9	M	FALSE
4	9	F	FALSE
5	10	M	FALSE
6	9	M	TRUE
7	3	M	TRUE
8	6	F	TRUE
9	9	M	TRUE
10	8	M	FALSE

Para saber si efectivamente un objeto en R es un DataFrame podemos ejecutar los siguientes comandos

```
class(datos)
```

```
[1] "data.frame"
```

```
is.data.frame(datos)
```

```
[1] TRUE
```

```
mode(datos)
```

```
[1] "list"
```

Nombres de columnas

Observe que los nombres de columna se toman de los nombres de los vectores usados para crear el data, si queremos otros nombres de columnas procedemos de la siguiente forma

```
datos=data.frame(Cx=x,Cy=y,Cz=z)
datos
```

	Cx	Cy	Cz
1	5	M	FALSE
2	9	M	FALSE
3	9	M	FALSE
4	9	F	FALSE
5	10	M	FALSE
6	9	M	TRUE
7	3	M	TRUE
8	6	F	TRUE
9	9	M	TRUE

Nombres de filas

También podemos colocar nombres de filas al data al momento de crearla, de la siguiente forma

```
datos=data.frame(Cx=x,Cy=y,Cz=z,row.names=month.abb[1:10])
datos
```

	Cx	Cy	Cz
Jan	5	M	FALSE
Feb	9	M	FALSE
Mar	9	M	FALSE
Apr	9	F	FALSE
May	10	M	FALSE
Jun	9	M	TRUE
Jul	3	M	TRUE
Aug	6	F	TRUE
Sep	9	M	TRUE
Oct	8	M	FALSE

Nombres de filas II

Cuando uno de los vectores con que creamos el data tiene nombres para sus elementos, la función `data.frame` toma como estos nombres como como nombres de fila del data creado.

```
names(y)=LETTERS[1:10]
datos=data.frame(Cx=x,Cy=y,Cz=z)
datos
```

	Cx	Cy	Cz
A	5	M	FALSE
B	9	M	FALSE
C	9	M	FALSE
D	9	F	FALSE
E	10	M	FALSE
F	9	M	TRUE
G	3	M	TRUE
H	6	F	TRUE

¿Qué pasa si mas de un vector tiene nombres? Veamos

```
names(y)=LETTERS[1:10]
names(x)=month.abb[1:10]
datos=data.frame(Cy=y,Cx=x,Cz=z,row.names=names(x))
datos
```

	Cy	Cx	Cz
Jan	M	5	FALSE
Feb	M	9	FALSE
Mar	M	9	FALSE
Apr	F	9	FALSE
May	M	10	FALSE
Jun	M	9	TRUE
Jul	M	3	TRUE
Aug	F	6	TRUE
Sep	M	9	TRUE
Oct	M	8	FALSE

Si no quiero nombres de fila se lo indico de la siguiente forma

```
datos=data.frame(Cy=y,Cx=x,Cz=z,row.names=NULL)
datos
```

	Cy	Cx	Cz
1	M	5	FALSE
2	M	9	FALSE
3	M	9	FALSE
4	F	9	FALSE
5	M	10	FALSE
6	M	9	TRUE
7	M	3	TRUE
8	F	6	TRUE
9	M	9	TRUE
10	M	8	FALSE

Nombres de filas, usos

Los nombres de filas se pueden usar para recuperar filas del data (indexamos el data con un vector de caracteres que contiene los nombres de las filas que quiere recuperar)

```
datos=data.frame(Cx=x,Cy=y,Cz=z)
datos[c('Jan','Mar','May'),]
```

	Cx	Cy	Cz
Jan	5	M	FALSE
Mar	9	M	FALSE
May	10	M	FALSE

Recuperación y cambio de nombres de filas y columnas

Para recuperar los nombres de filas de un data usamos la función `row.names` o `rownames` y para las columnas usamos `colnames`, de la siguiente forma

```
# filas  
row.names(datos)
```

```
[1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct"
```

```
rownames(datos)
```

```
[1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct"
```

```
# columnas  
colnames(datos)
```

```
[1] "Cx" "Cy" "Cz"
```

Recuperar nombres de columnas

```
# columnas  
colnames(datos)
```

```
[1] "Cx" "Cy" "Cz"
```

Para cambiar los nombres de columnas procedemos de la siguiente forma

```
colnames(datos)=c('Colx','Coly','Colz')
datos
```

	Colx	Coly	Colz
Jan	5	M	FALSE
Feb	9	M	FALSE
Mar	9	M	FALSE
Apr	9	F	FALSE
May	10	M	FALSE
Jun	9	M	TRUE
Jul	3	M	TRUE
Aug	6	F	TRUE
Sep	9	M	TRUE
Oct	8	M	FALSE

Cambiar el nombre de una sola columna

```
colnames(datos)[2]='Col2'
datos
```

	Colx	Col2	Colz
Jan	5	M	FALSE
Feb	9	M	FALSE
Mar	9	M	FALSE
Apr	9	F	FALSE
May	10	M	FALSE
Jun	9	M	TRUE
Jul	3	M	TRUE
Aug	6	F	TRUE
Sep	9	M	TRUE
Oct	8	M	FALSE

Creación de nuevas columnas

Podemos crear nuevas columnas en un data frame, a partir de las columnas existentes, de la siguiente forma

```
datos$Cx2=datos$Colx^2
datos
```

	Colx	Col2	Colz	Cx2
Jan	5	M	FALSE	25
Feb	9	M	FALSE	81
Mar	9	M	FALSE	81
Apr	9	F	FALSE	81
May	10	M	FALSE	100
Jun	9	M	TRUE	81
Jul	3	M	TRUE	9
Aug	6	F	TRUE	36
Sep	9	M	TRUE	81
Oct	8	M	FALSE	64

Crearemos una columna que contiene la columna Col2 codificada de la siguiente forma: Si Col2 es M asignamos 1 (uno), si es F asignamos 0 (cero).

```
datos["Sex"]=ifelse(datos$Col2=="M",1,0)
datos
```

	Colx	Col2	Colz	Cx2	Sex
Jan	5	M	FALSE	25	1
Feb	9	M	FALSE	81	1
Mar	9	M	FALSE	81	1
Apr	9	F	FALSE	81	0
May	10	M	FALSE	100	1
Jun	9	M	TRUE	81	1
Jul	3	M	TRUE	9	1
Aug	6	F	TRUE	36	0
Sep	9	M	TRUE	81	1
Oct	8	M	FALSE	64	1

Borrado de columna

Para borrar una columna usamos NULL de la siguiente forma

```
datos$Cx2=NULL
datos
```

	Colx	Col2	Colz	Sex
Jan	5	M	FALSE	1
Feb	9	M	FALSE	1
Mar	9	M	FALSE	1
Apr	9	F	FALSE	0
May	10	M	FALSE	1
Jun	9	M	TRUE	1
Jul	3	M	TRUE	1
Aug	6	F	TRUE	0
Sep	9	M	TRUE	1
Oct	8	M	FALSE	1

Indexando Data Frames

Como con los vectores y matrices, podemos indexar un data frame usando como indices

- Enteros positivos
- Enteros negativos
- Valores lógicos
- Caracteres

Esto es válido tanto para filas como para columnas

Veamos algunos ejemplos

```
datos[c(1,5,7,9,10),c(2,3)]
```

	Col2	Col3
Jan	M	FALSE
May	M	FALSE
Jul	M	TRUE
Sep	M	TRUE
Oct	M	FALSE

Indexado con numeros negativos

```
datos[-c(1,5,7,9,10),-c(2,3)]
```

	Colx	Sex
Feb	9	1
Mar	9	1
Apr	9	0
Jun	9	1
Aug	6	0

Ejemplo de indexado con caracteres

```
datos[c('Feb', 'Apr', 'Jun', 'Aug'), -1]
```

	Col12	Colz	Sex
Feb	M	FALSE	1
Apr	F	FALSE	0
Jun	M	TRUE	1
Aug	F	TRUE	0

Indexación lógica

En este tipo de indexación se extraen las filas o columnas que coincidan con un valor TRUE , veamos ejemplos

```
datos[c(TRUE,FALSE,TRUE,FALSE,TRUE,FALSE,TRUE,FALSE,TRUE,FALSE,
```

	Colx	Col2	Colz	Sex
Jan	5	M	FALSE	1
Mar	9	M	FALSE	1
May	10	M	FALSE	1
Jul	3	M	TRUE	1
Sep	9	M	TRUE	1

```
datos[!c(TRUE,FALSE,TRUE,FALSE,TRUE,FALSE,TRUE,FALSE,TRUE,FALS
```

	Colx	Col2	Colz	Sex
Feb	9	M	FALSE	1
Apr	9	F	FALSE	0
Jun	9	M	TRUE	1

Ejercicio

Ejercicio: Calcular la media de Colx pero para aquellas filas donde Col2 es igual a "M" y separadamente para donde Col2 es igual a "F"

```
datosM=datos[datos$Col2=="M",]  
mean(datosM$Colx)
```

```
[1] 7.75
```

Para los femeninos, es decir, donde Col2 es igual a "F"

```
datosF=datos[datos$Col2=="F",]  
mean(datosF$Colx)
```

```
[1] 7.5
```

Otra forma, usando la negación

```
datosF=datos[!datos$Col2=="M",]  
mean(datosF$Colx)
```

```
[1] 7.5
```

Ejercicio

Calcular la varianza de Colx para aquellas filas donde Colz es TRUE

```
var(datos[datos$Colz,][["Colx"]])
```

```
[1] 8.25
```

Ejercicio

Calcular la varianza de Colx para aquellas filas donde Colz es FALSE

```
var(datos[!datos$Colz,][["Colx"]])
```

```
[1] 3.066667
```

Ejercicio

Calcular la media de Colx para aquellas filas donde Col2 es 'M' y Colz es FALSE

```
mean(datos[datos$Col2=='M' & !datos$Colz, ][['Colx']])
```

```
[1] 8.2
```

Ejercicio

Cuántos hombres, es decir Col2=='M' tienen un valor de Colx por encima de la media de Colx

```
nrow(datos[datos$Colx>mean(datos$Colx) & datos$Col2=="M" ,])
```

```
[1] 6
```

otra forma

```
sum(datos$Colx > mean(datos$Colx) & datos$Col2 == "M")
```

```
[1] 6
```

otra forma

```
sum(datos[datos$Colx>mean(datos$Colx),][['Col2']]=="M" )
```

```
[1] 6
```